Лабораторная работа № 5. Кэширование данных с Redis

Цель работы: проверить, как Redis влияет на задержку, число обращений к медленному источнику и устойчивую пропускную способность приложения.

1. Приложение и окружение

Я развернул отдельные cache-app и Redis в namespace vp-lr5 прежнего K3s. GET /data/<item_id> без кэша каждый раз ждёт 50 мс и возвращает значение. При cache=on приложение сначала проверяет Redis, при промахе обращается к тому же источнику и сохраняет ответ с TTL. Это стратегия Cache-Aside. Один процесс Gunicorn обрабатывает запросы последовательно. Во всех сериях код, ресурсы Pod и путь через NodePort 30085 одинаковые.

Зависимости контейнера взяты из официального PyPI и сохранены локально: внутри Pod установка из сети не работала. Исходные файлы и порядок запуска приведены в README. Приложение отдаёт /metrics, Prometheus собирает его счётчики и гистограмму.

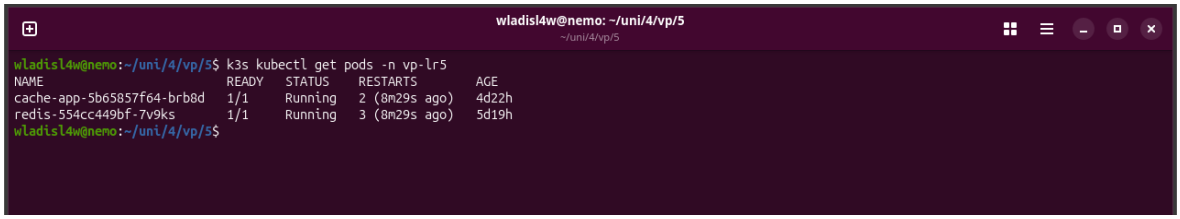


Рисунок 1. Работающие Pod ЛР5 в терминале Ubuntu

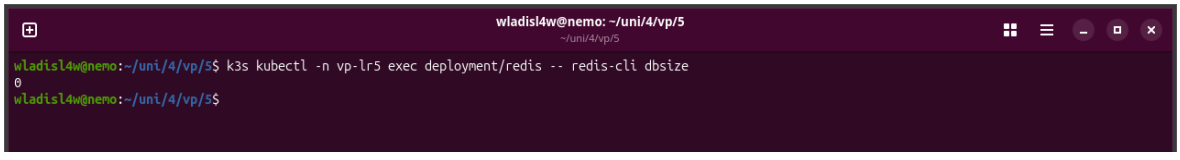


Рисунок 2. Проверка Redis после истечения TTL

2. Методика тестирования

Один сценарий k6 задавал постоянную интенсивность запросов. Идентификаторы перебирались по кругу, поэтому при одинаковом числе ключей оба режима получали одни и те же объекты. Для основного сравнения использованы цели 10, 25, 50, 100, 200 и 400 запросов/с по 60

секунд. Перед каждым прогоном учебный Redis очищался. В режиме cache=on для сравнения пропускной способности я предварительно заполнял десять ключей; TTL оставался 30 секунд. Результаты k6, показания счётчиков до и после прогона и ответы Prometheus сохранены отдельно в results.

Устойчивым считаю уровень с p95 меньше 500 мс, ошибками меньше 1%, без dropped_iterations и с фактическим RPS не ниже 95% цели. Это оценка только проверенной сетки и длительности теста.



Рисунок 3. Просмотр сохранённого вывода k6 без кэша при цели 50 запросов/с

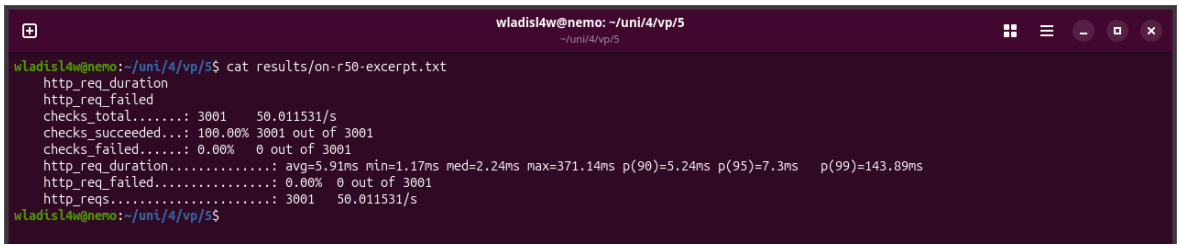


Рисунок 4. Просмотр сохранённого вывода k6 с Redis при той же цели

3. Сравнение без кэша и с Redis

Таблица 1. Результаты при одинаковой целевой интенсивности

Режим	Цель, RPS	Факт, RPS	p50, мс	p95, мс	p99, мс
off	10	10.01	51.92	52.89	54.45
on	10	10.00	2.11	4.37	53.10
off	25	19.67	7084.08	10000.52	10000.83
on	25	25.02	2.00	4.02	92.06
off	50	37.26	10000.04	10001.37	10003.24
on	50	50.01	2.25	7.31	143.90
off	100	52.01	9999.91	10002.38	10005.60
on	100	100.01	1.52	3.66	218.13
off	200	54.19	10000.02	10000.87	10003.13
on	200	199.95	1.92	7.83	316.34
off	400	54.11	9999.74	10005.31	10010.01
on	400	392.86	2.02	225.43	345.99

Таблица 2. Ошибки и пропущенные итерации при тех же нагрузках

Режим	Цель, RPS	Ошибки, %	Dropped
off	10	0.00	0
on	10	0.00	0
off	25	18.23	124
on	25	0.00	0
off	50	85.58	393
on	50	0.00	0
off	100	93.05	2360
on	100	0.00	0
off	200	94.12	8210
on	200	0.00	2
off	400	94.61	20213
on	400	0.00	78

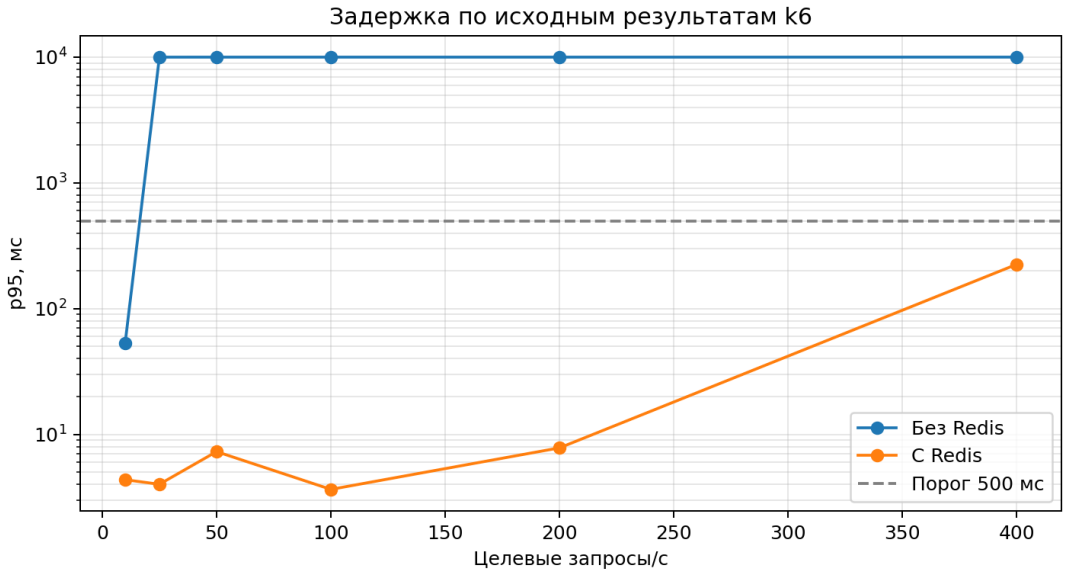


Рисунок 5. Сравнение p95 по реальным результатам k6

Таблица 3. Максимальные измеренные устойчивые уровни

Режим	Цель, RPS	Факт, RPS	p95, мс	Рост RPS
Без кэша	10	10.01	52.89	1,00
С Redis	100	100.01	3.66	9.99

Коэффициент роста устойчивого RPS я считал как RPS с Redis / RPS без Redis: 9.99. В методичке на странице с формулой числитель и знаменатель записаны в обратном порядке, поэтому для роста я использовал это отношение.

Задержка без Redis включает имитацию обращения к источнику в 50 мс. При попадании в Redis этот вызов не выполняется. Рост RPS следует оценивать вместе с p95, ошибками и пропущенными итерациями, а не по числу отправленных запросов.

При одинаковой цели 10 запросов/с p95 без кэша составил 52.89 мс, с Redis - 4.37 мс. Отношение задержек без Redis / с Redis равно 12.11. В прогретой серии обращения к источнику уменьшились с 601 до 20 за один опыт.

4. Разнообразие ключей и эффективность кэша

При одинаковых 10 запросах/с и TTL 30 секунд я проверил 10, 100 и 1000 разных объектов. Опыт с 1000 ключами длился 120 секунд, остальные - 60 секунд: так каждый из 1000 объектов действительно мог быть запрошен. Перед каждой серией Redis очищался, поэтому первые запросы к ключам были промахами. Доля попаданий рассчитана как hits / (hits + misses) по разности счётчиков до и после прогона. Source RPS рассчитан из числа обращений к источнику за фактическое время запуска k6.

Таблица 4. Влияние числа разных объектов

Ключей	Hit Ratio, %	p95, мс	Source RPS	Ключей в Redis
10	96.67	4.85	0.33	10
100	66.67	53.99	3.29	100
1000	0.00	55.13	9.97	301

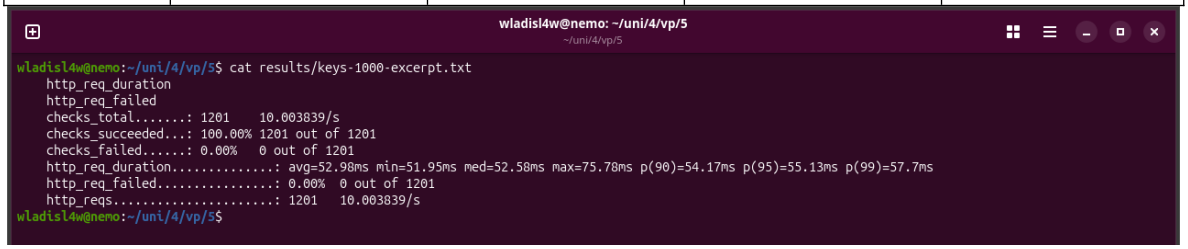


Рисунок 6. Просмотр сохранённого вывода k6 для 1000 объектов

При 10 ключах доля попаданий составила 96.67%, а при 1000 ключах - 0.00%. Чем реже повторяется один объект за время TTL, тем меньше пользы от кэша. Длительность серий различалась, поэтому числа обращений к источнику для этого сравнения я перевёл в RPS.

После опыта с 1000 объектами команда DBSIZE показывала 301 ключей. Это число активных записей на момент проверки, а не число запрошенных объектов: старые записи успели истечь при TTL 30 секунд.

5. Влияние TTL

При 10 ключах и одинаковой цели 10 запросов/с я сравнил TTL 5, 30 и 120 секунд. Каждый опыт длился 150 секунд, чтобы даже запись с TTL 120 секунд могла истечь. Перед каждым опытом Redis очищался. Большее время жизни уменьшает повторные обращения к источнику, но увеличивает время возможного устаревания значения.

Таблица 5. Измерения при разном TTL

TTL, с	Hit Ratio, %	Обращений к источнику	Source RPS	p95, мс
5	83.33	250	1.66	53.62
30	96.67	50	0.33	4.79
120	98.67	20	0.13	13.72

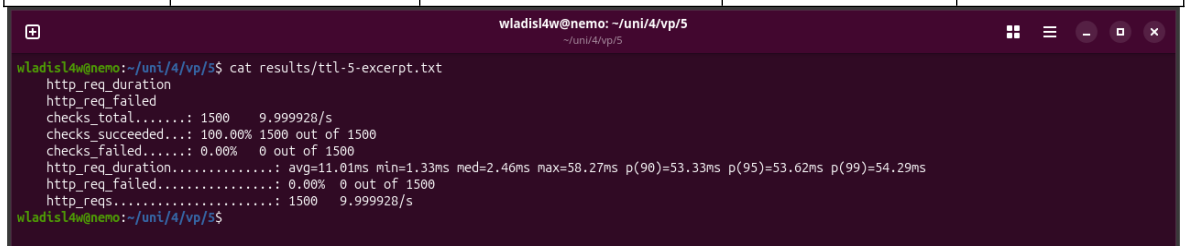


Рисунок 7. Просмотр сохранённого вывода k6 при TTL 5 секунд

За 150 секунд при TTL 5 с источник получил 250 обращений, при TTL 120 с - 20. Больший TTL экономит обращения, если повторные запросы приходят раньше истечения записи; ценой становится возможное устаревание ответа.

6. Анализ в Grafana

Dashboard показывает HTTP RPS и p95 по режимам, Hit Ratio, обращения к источнику в секунду, попадания и промахи. На снимках открыт сохранённый период реальных испытаний. При cache=off счётчики попаданий не растут: в этом режиме приложение сразу обращается к источнику.

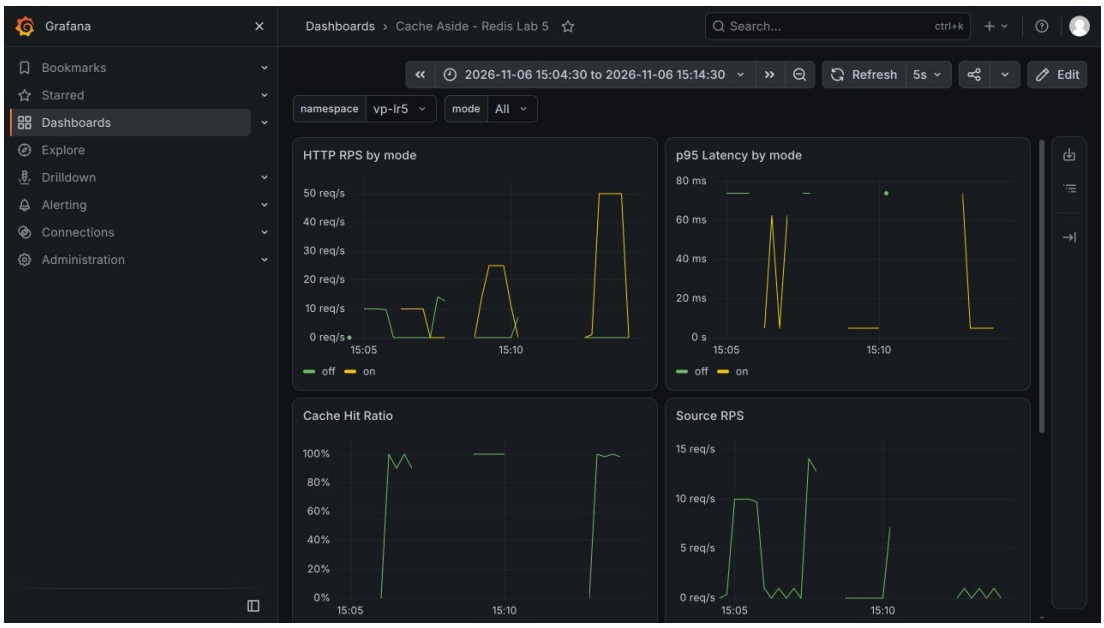


Рисунок 8. Grafana: RPS и p95 за период сравнения режимов

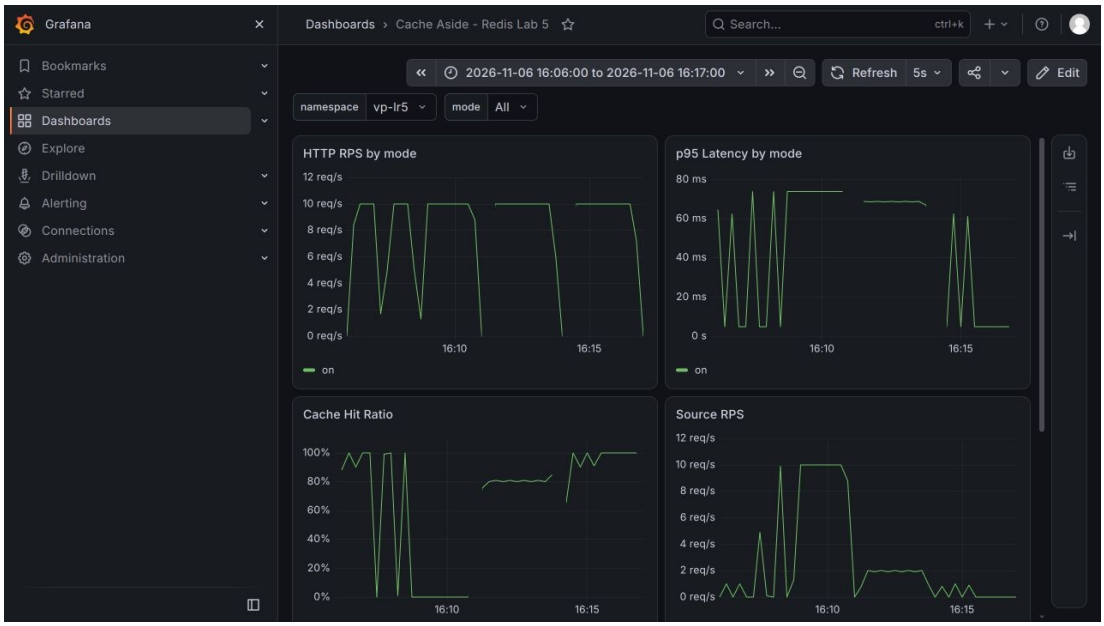


Рисунок 9. Grafana: Hit Ratio и Source RPS за период исследования кэша

Наибольший выигрыш Redis даёт при повторных обращениях к небольшому набору объектов до истечения TTL: тогда Hit Ratio высок, Source RPS низок, а p95 остаётся меньше порога даже при увеличении HTTP RPS. При большом разнообразии ключей промахов становится больше, поэтому обращения к источнику и задержка растут. Кратковременные провалы графика при перегрузке связаны с тем, что однопроцессное приложение поздно отвечало и сборщику метрик; для итоговых чисел я использовал исходные результаты k6.

Вывод

Я реализовал Cache-Aside через Redis и измерил работу в двух режимах. В проверенной сетке наибольший устойчивый уровень без кэша составил 10.01 RPS, с Redis - 100.01 RPS. Данные по числу ключей и TTL показывают, как попадания в кэш снижают обращения к источнику. Результат относится к одному процессу приложения и выбранной нагрузке; для более точного предела нужны дополнительные уровни между соседними проверенными точками.